VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi, Karnataka-590018

A MINI PROJECT REPORT
ON
AI Call Agent

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by

4CB23CS072 Kartik Mahesh Kamat
4CB23CS085 Mahalaxmi Mangesh Shanbhag
4CB23CS095 Mukund Bhaskar Pai
4CB23CS099 Nandita Naik

Under the Guidance of

Mr. Adithya M

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CANARA ENGINEERING COLLEGE

SUDHINDRA NAGARA, BENJANAPADAVU, BANTWAL-574219

2025-2026

CANARA ENGINEERING COLLEGE
BANTWAL, D.K. 574219 – KARNATAKA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the mini project work entitled **AI Call Agent** carried out by **Mr.Kartik Mahesh Kamat(4CB23CS072)**, **Mrs.Mahalaxmi Mangesh Shanbhag(4CB23CS085)**, **Mr.Mukund Bhaskar Pai(4CB23CS095)**, and **Mrs.Nandita Naik(4CB23CS099)**, a bonafide student of**CANARA ENGINEERING COLLEGE, BENJANAPADAVU** in partial fulfillment for the award of **BACHELOR OF ENGINEERING** in **COMPUTER SCIENCE AND ENGINEERING** of the **VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI** during the year **2025 - 2026**. The project report has been approved as it satisfies the academic requirements in respect of Mini Project work prescribed for the said Degree.

Signature **Project Guide**	**Signature** **Head of Department**

ABSTRACT

The AI Call Agent serves as an intelligent communication system, which automatizes real-time voice conversations between users and technicians. STT, NLP, and TTS technologies, integrated in this system, allow it to rule out the manual handling of calls and hence cut down on delays in communication. The agent initiates outbound calls, understands the queries of the user, recognizes intent, and sends responses through artificial intelligence-natural speech, thereby making the process of assigning technicians much quicker and enhancing service efficiency.

The system uses FastAPI for back-end integration, Twilio for call management, Whisper for speech recognition, and advanced NLP models for intent classification and controlling the flow of dialogue. This automation enhances customer satisfaction by providing support on a 24x7 basis, reducing human workload, and offering consistent and accurate responses. Besides this, the AI Call Agent aggregates and analyzes call data to support the identification of frequent issues and to enable better decision-making. In general, the project demonstrates how AI-driven voice automation could facilitate communication processes and transform traditional technician support systems.

ACKNOWLEDGEMENT

It is our great pleasure to acknowledge the assistance and contributions of all the people who helped us to make our Mini Project successful.

We wish to express our deepest gratitude to project guide **Prof. Adithya M**, **Department of Computer Science and Engineering, CEC, Mangalore**, for their invaluable support, guidance, and encouragement throughout the duration of this project.

We extend our thanks to **Prof. Saritha Suvarna**, **Project Coordinator, Department of Computer Science & Engineering, CEC, Mangalore**, for their exceptional support and coordination throughout this project.

We are extremely grateful to **Dr.Karthik Pai B H,Professor and Head of the Computer Science & Engineering Department, CEC**, Mangalore, for his moral support and valuable suggestions throughout this project.

We thank **Dr. Demian Antony D'Mello, Vice Principal, CEC, Mangalore**, for his valuable suggestions throughout this project.

We would like to express our deepest gratitude to our **Principal, Dr. Nagesh H R** whose unwavering support and guidance have been instrumental in the successful completion of this project.

We thank all the faculty and technical staff of **Department of Computer Science Engineering** for their kind help.

Kartik Mahesh Kamat	4CB23CS072
Mahalaxmi Mangesh Shanbhag	4CB23CS085
Mukund Bhaskar Pai	4CB23CS095
Nandita Naik	4CB23CS099

TABLE OF CONTENTS

CHAPTERS	PAGE NO
1 Introduction	1
1.1 Overview	1
1.2 Problem Statement	3
1.3 Scope of the Project	3
2 Literature Survey	4
3 Software Requirements Specification	6
3.1 Introduction	6
3.1.1 Purpose	6
3.1.2 Definitions, Acronyms and Abbrevations	6
3.2 The Overall Description	7
3.2.1 Product Perspective	7
3.2.2 Operating Environment	8
3.3 Specific Requirements	8
3.3.1 Hardware Requirements	9
3.3.2 Software requirements	9
3.4 Functional requirements	9
3.5 Non functional requirements	9
4 System Design	11
4.1 Architectural Design	11
4.2 Use Case Diagram	13
4.3 Activity Model	15
5 IMPLEMENTATION AND TESTING	17
5.1 Implementation	17
5.2 Testing	18
5.2.1 Unit Testing	18
5.2.2 Integration Testing	19
5.2.3 System Testing	19
6 Results and Analysis	20
7 Conclusion	25

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
4.1	System Architecture Design	13
4.2	Use Case Diagram	15
4.3	Activity Model	16
6.1	Create Account Page	21
6.2	Login Page - User Authentication Screen	22
6.3	AI Call Agent- Home Dashboard Interface	22
6.4	Technician Management Panel	23
6.5	Conversation View–Ticket Interaction Details	23
6.6	AI Call Agent-Conversation and Live Transcription	24

Chapter 1

Introduction

1.1 Overview

AI Call Agent is an intelligent automated system to handle real-time voice conversations between users and technicians. The traditional call-handling process predominantly involves manual operations, which lead to delays, miscommunication, and a slower response time. The advanced technologies used, such as Speech-to-Text, NLP, and Text-to-Speech in AI Call Agent, provide faster, more reliable, and fully automated communication channels.

Applications for parent-teacher communication connect the gap between home and school by providing an interactive, one-stop platform. Parents can get real-time information on the child's academic progress, attendance, behavior, and forthcoming events in school. Teachers can also instantly share information, send reminders, and provide resources to support learning outside the classroom. Such immediacy and ease ensure the parents are always informed, limiting missed information and enabling timely intervention when necessary.

This project showcases the potential of AI to change traditional support systems and help reduce response delays with improved customer satisfaction. With modern AI tools along with voice-processing frameworks, the AI Call Agent will serve as a strong solution toward automating technical support interactions and ensuring better workforce productivity.

With an ever-growing volume of customer queries and a growing demand for instant technical support, the application of intelligent automation in communication systems has become highly demanding in recent years. Most call centers face difficulties in maintaining

consistent service quality due to the high volume of work, human error, and limited availability. The AI Call Agent is designed to address these issues by providing a scalable and efficient alternative capable of handling multiple calls simultaneously. By automating repetitive communication tasks, it reduces operational costs and ensures timely assistance for customers without long wait periods.

Moreover, developments in machine learning and speech technologies have made the AI-driven communication way more natural and reliable. The AI Call Agent uses state-of-the-art speech recognition models, full NLP pipelines, and high-quality TTS engines that together yield a smooth and natural-feeling call experience. Capable of understanding spoken language in real time and responding to it meaningfully, seamless interactions are achieved that closely resemble human-to-human conversations. The system is therefore appropriate for several real-world applications such as customer support, service coordination, and technical troubleshooting, demonstrating the transformative impact of AI in modern communication infrastructures.

1.2 Problem Statement

As such, traditional technician communication depends on outbound calling manually, creating delays, miscommunication, and decreased productivity. This leads to increased customer wait times and reduced issue resolution rates. Therefore, there is a need for an intelligent AI-powered system that can automatically connect technicians, manage real-time conversations, and update service systems efficiently.

1.3 Scope of the Project

- Includes real-time outbound call automation for technicians.
- Covers speech-to-text, intent detection, and the generation of AI responses.
- Focus on the dialogue flow management using NLP.
- Does not include hardware-level telephony or physical network infrastructure.

Chapter 2

Literature Survey

John Luther et al. **"AI Call Agent"** This paper focuses on developing an AI-based agent to handle customer service interactions. It uses natural language processing and general machine learning algorithms to improve responsiveness and consistency. The system enhances customer satisfaction by providing timely and uniform support [1].

Smith Seth et al. **"Smart Call Routing"** The study explores intelligent routing mechanisms to connect callers with the most suitable resources. It combines decision trees for transparent rule-based logic and neural networks for complex prediction tasks. The result is reduced wait times and more accurate agent-customer matching [2].

Lee Jiyeon et al. **"Real-time Conversation Management"**This work addresses the challenge of managing live customer conversations with AI guidance. It applies deep learning for contextual understanding and NLP for intent detection and dialogue control. The system improves conversational flow, reduces interruptions, and maintains context throughout interactions [3].

KumarGaurav et al. **"Service System Efficiency"**The paper aims to enhance operational efficiency across service systems. It leverages predictive analytics and machine learning to forecast demand and allocate resources effectively. The outcome is better staffing decisions and proactive service delivery [4].

S.R.Bharamagoudar, Geeta R.B, S.G.Totad. et al. **"AI in Customer Support "**This research integrates AI into multi-channel customer support platforms. It utilizes chatbots and NLP to automate FAQs and guide users through support flows. The system ensures

smoother transitions between channels and delivers consistent responses across platforms [5].

Brown et al. **"Automated Call Handling"**The study automates the entire call handling process, from greeting to resolution. It employs voice recognition to detect caller intent and machine learning to select appropriate actions. The approach minimizes human involvement, allowing agents to focus on complex cases [6].

Wilson et al. **"AI-driven Feedback Analysis "**This paper analyzes customer feedback to extract sentiment and thematic insights. It uses NLP and sentiment analysis on call transcripts and surveys. The findings help organizations understand customer needs and improve products and services accordingly [7].

Patel et al. **"Intelligent Call Distribution"**The research proposes a smarter system for distributing incoming calls. It combines rule-based logic with machine learning to adapt routing strategies over time. The system enhances agent utilization and overall distribution efficiency [8].

Zhang et al. **"Conversational AI for Escalation Prediction"**This work focuses on predicting when a customer interaction should be escalated to a human agent. It uses sequence modeling techniques like RNNs and attention mechanisms. The system reduces unnecessary escalations and improves resolution accuracy [9].

Fernandez et al. **"Multilingual AI Support Systems "**The paper develops AI systems capable of handling customer service in multiple languages. It applies transfer learning with multilingual BERT models and integrates language detection and translation pipelines. The solution enables consistent support across global markets [10].

Chapter 3

Software Requirements Specification

3.1 Introduction

Software Requirement Specification, SRS, is a definition of the behavior of the projected software without detailing how the software will perform its operations. Software requirements are the conditions or capabilities that users must have in order to solve problems or achieve objectives. This section provides insight into hardware and software requirements and an overview of the proposed framework.

3.1.1 Purpose

This project outlines the specifications for the *AI Call Agent* system, to be used as an intelligent, voice-driven first line of IT support. The system automates the first level of diagnosis and guides users through issue reporting using natural, conversational-style interaction.

By incorporating advanced technologies, such as STT, TTS, NLP, and LLMs, the system guides the user through troubleshooting, creates or updates support tickets, and escalates critical issues when needed. It will communicate seamlessly with the technicians via a Twilio-powered IVR that not only conveys details of the issue but also helps schedule an appointment.

This application is intended to bring efficiency in IT support, quicker response times, and satisfaction in users by means of automation, real-time interaction, and reliable mechanisms of escalation.Strong security including user authentication and API key protection will guarantee data privacy and integrity.

3.1.2 Definitions, Acronyms and Abbrevations

- AI: Artificial Intelligence
- IVR: Interactive Voice Response

- STT: Speech-to-Text (speech transcription)
- TTS: Text-to-Speech - Speech synthesis
- NLP: Natural Language Processing
- LLM:Large Language Model.
- SPA: Single-Page Application
- GPU: Graphics Processing Unit
- API: Application Programming Interface
- SMS: Short Message Service (text messaging)

3.2 The Overall Description

The AI Call Agent is a client–server application designed to automate and manage voice-based customer interactions. The back-end integrates telephony, speech-to-text, text-to-speech, and AI-driven dialogue processing through a range of cloud services including Twilio, Whisper, Hugging Face, Perplexity, Gemini, and OpenAI. The frontend is a browser-based Single Page Application (SPA) that offers real-time monitoring of tickets, technicians, and IVR logs to administrators. The application runs continuously on an active internet connection for interaction with the required cloud APIs.The various key features are:

3.2.1 Product Perspective

The AI Call Agent acts as an integrated communication system in a bigger customer support or IT service setup. It interacts with various external components, including:

- Telephony providers, such as Twilio, for inbound and outbound calls
- Conversation understanding AI/ML platforms: OpenAI, Gemini, Hugging Face and Perplexity
- Speech services for STT/TTS: Whisper, cloud TTS engines

- Ticketing or CRM systems for logging and resolving support issues
- Admin Dashboard for the SPA frontend: monitor calls, status of tickets, technician activity, IVR analytics

  In such a landscape, the product acts as an intelligent automation layer, which receives the calls coming in, processes speech, generates responses, and routes information to backend databases and dashboards.

3.2.2 Operating Environment

The application is designed to run in a flexible cross-platform environment:

Backend

- Runs on Windows, macOS or Linux
- Built in Python
- May use GPU acceleration for faster local transcription - NVIDIA CUDA, ROCm among others
- Requires continuous internet access to interact with cloud APIs
- Integrates with external servers/services via REST or WebSocket interfaces

Frontend

- Web-based Single Page Application (SPA)
- Accessible through all modern browsers: Chrome, Firefox, Edge, and Safari
- Deployed on a web server or cloud hosting platform.

3.3 Specific Requirements

This section contains all of the functional and quality requirements of the system. It gives detailed description of the system and all its features.

3.3.1 Hardware Requirements

- Hardware -(Server): Multi-core CPU, 8GB RAM minimum; NVIDIA GPU recommended for optimal STT..
- Hardware (Client): standard computer with microphone and speakers/headphones.

3.3.2 Software requirements

- Software (Backend): Python 3.x, dependencies in requirements.txt, optional ngrok for webhook exposure.
- Software (Client): Modern browser supporting Web Audio and MediaRecorder APIs.

3.4 Functional requirements

1. Secure authentication with login/signup.
2. Real-time voice conversation with STT transcription.
3. Diagnosis through automation, using NLP and LLM prompting.
4. Creating or updating support tickets.
5. Automated escalation via a Twilio IVR call.
6. Scheduling of technician appointments by voice.
7. SMS notifications for technicians.
8. Administrator dashboard for technician management and IVR log monitoring.

3.5 Non functional requirements

1. Performance: The performance is measured by how accurately the application predicts the information
2. Reliability: The application must perform consistent prediction
3. Availability: The application must be able to predict the outcome at any instance of time

4. Maintainability: The application is simple, easy to use and user friendly
5. Portability: Currently the project is developed for prediction of diabetes but can be developed for preventing other related diseases

Chapter 4

System Design

This document represents the system-level design for the AI Call Agent project, presented in three parts: Architectural design, Use case design, and Activity models. Implementation, documentation, and diagrams - which can be produced in PlantUML or Draw.io from these textual models - are all based on this authoritative reference.

4.1 Architectural Design

AI Call Agent is a client-server application with clear separation of concerns between a frontend, a backend, and integrations with external services. The frontend is a browser-based SPA that captures user audio, plays audio of AI-generated responses, reflects the states of a ticket and an appointment, and provides a login for the user. The backend, written in Python and using the Flask framework, is responsible for speech transcription, intent extraction, the dialogue state machine, telephony logic, scheduling, and database operations. It orchestrates several services, including Twilio for calls and SMS, LLM providers for reasoning, TTS engines to generate audio, and STT services, such as Whisper or faster-whisper on GPU.

Logically, the system is divided into a number of components. The Web Client contains features such as the VoiceChat module powered by MediaRecorder and VAD, transcript panels, waveform displays, timelines, and authentication pages. It communicates with the backend via REST APIs or WebSockets. The API server exposes endpoints such as /conversation, /tickets, /twilio-ivr, /auth, and /technicians. It receives audio, routes it through the STT pipeline, and invokes the AIAgent for NLP reasoning. The backend manages Twilio interactions by emitting TwiML responses, and it handles TTS conversion through

pluggable adapters. The persistence layer uses SQLAlchemy models such as Ticket, Technician, CallLog, and Appointment, supported by SQLite for development and Postgres for production. The AIAgent will contain all domain logic, including the conversation state machine, entity normalization pipeline, LLM orchestrator, diagnostic checks, and escalation logic. Both STT and TTS services are designed to be pluggable providers, supporting local and cloud-based models, automatically falling back.

Physically, the system runs on multiple nodes. This would include a client browser that communicates with the backend server over secure HTTPS. The backend can optionally run with GPU support, faster-whisper, and communicates with the outside world to Twilio, STT APIs, LLMs, and TTS providers. Twilio interfaces with both the backend webhook endpoints and the public telephone network to provide outbound calls and SMS alerts. A database of either local SQLite or cloud Postgres persists tickets, logs, appointments, and IVR events. The network flow includes several HTTPS connections: browser backend, backend cloud AI services, and backend Twilio REST APIs.

A typical sequence of escalation would be initiated when the user expresses urgency. The AIAgent escalation flags the issue and chooses a technician based on availability and skill. The backend triggers a Twilio outbound call to the technician using the webhook URL attached to the Call SID. Twilio then routes the call, requesting TwiML instructions from the backend, and speaks an announcement describing the issue. It gathers the technician's input, sends the recording to the backend, and the server processes the transcription, determining the technician's availability. The system updates the appointment, sends an SMS confirmation, logs the event, and notifies the frontend interface. Resiliency, security, observability, and scalability are designed as cross-cutting concerns. Provider fallbacks are implemented in the system for STT, TTS, and LLM services in case of provider failures, along with retry and exponential backoff strategies against network failures. Security is provided by HTTPS, environment-managed API keys, JWT/OAuth authentication, and

role-based access. Structured JSON logging, audit logs for IVR actions, and metrics for performance analysis are some of the measures included to establish observability. The architecture allows for horizontal scaling with stateless API processes and can include task queues like Celery for heavy workloads, such as large audio transcription.

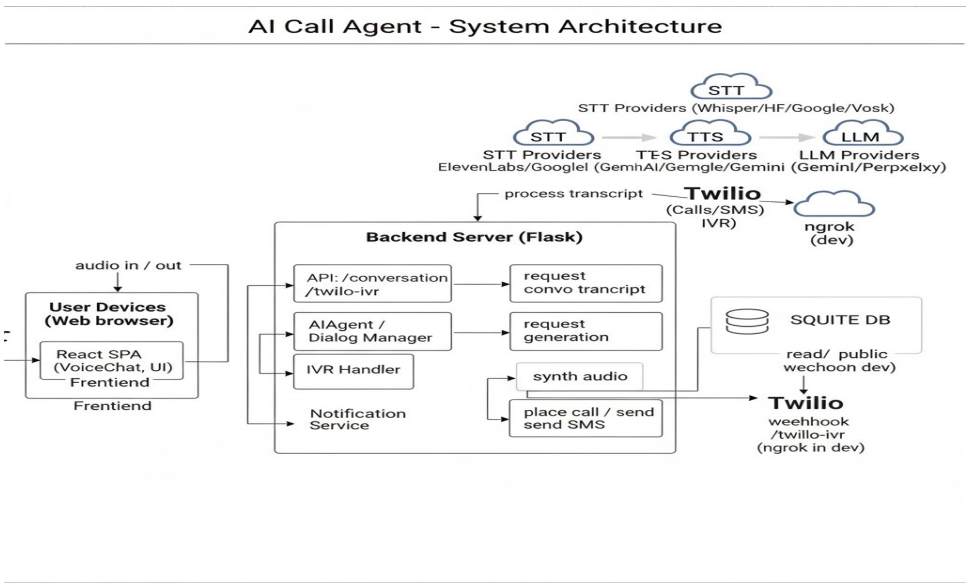

Figure 4.1: System Architecture Design

4.2 Use Case Diagram

The AI Call Agent system works with several types of people and services. First, the End User is the person who needs help—they either call through Twilio or talk to the AI using their web browser. The AI Agent is the main system that talks to users, understands what they mean, figures out the problem, and decides whether it can fix the issue or needs to call a human. A Technician steps in when the problem requires a real person; they get an automated IVR call, tell the system when they are available, and confirm the appointment. An Administrator manages the system by adding or editing technician details, updating skills, and checking logs or past calls. The system also depends on External Providers—speech-to-text, language models, text-to-speech, and telephony services—to recognize speech, understand language, generate replies, and make phone calls.

The system handles a few main tasks. In UC-01: Submit Problem via Voice, the user presses Record and explains their issue. The system converts their voice into text, creates a support ticket, and starts analysing the problem. In UC-02: Automated Diagnosis, the AI talks back and forth with the user, asking questions and suggesting simple fixes. If the issue can't be solved automatically, the system moves to escalation. In UC-03: Escalate and Schedule Technician, the AI calls a technician through an automated voice call, explains the problem, asks when they are free, listens to their response, converts it to text, and schedules the appointment. After that, UC-04: Technician Confirmation via SMS sends the technician a text message confirming the appointment details. In UC-05: Admin Manages Technicians and Logs, the admin can update technician information, add new ones, and check system logs through a dashboard.

A summary of these use cases shows how things normally work. In UC-01, the flow starts when the user clicks the Record button after logging in. The system creates a ticket and replies with an acknowledgement. In UC-03, escalation happens when the AI decides the issue needs a human or the user asks for a technician. If the right technician is available, the system calls them using Twilio, schedules the appointment based on their input, and updates the ticket to "Scheduled." If you drew a use case diagram, you would show all the actors—users, technicians, admins, and external service providers—outside the main box labeled AI Call Agent, and inside the box, you'd place all the major system actions like ticket creation, diagnosis, scheduling, and admin functions.

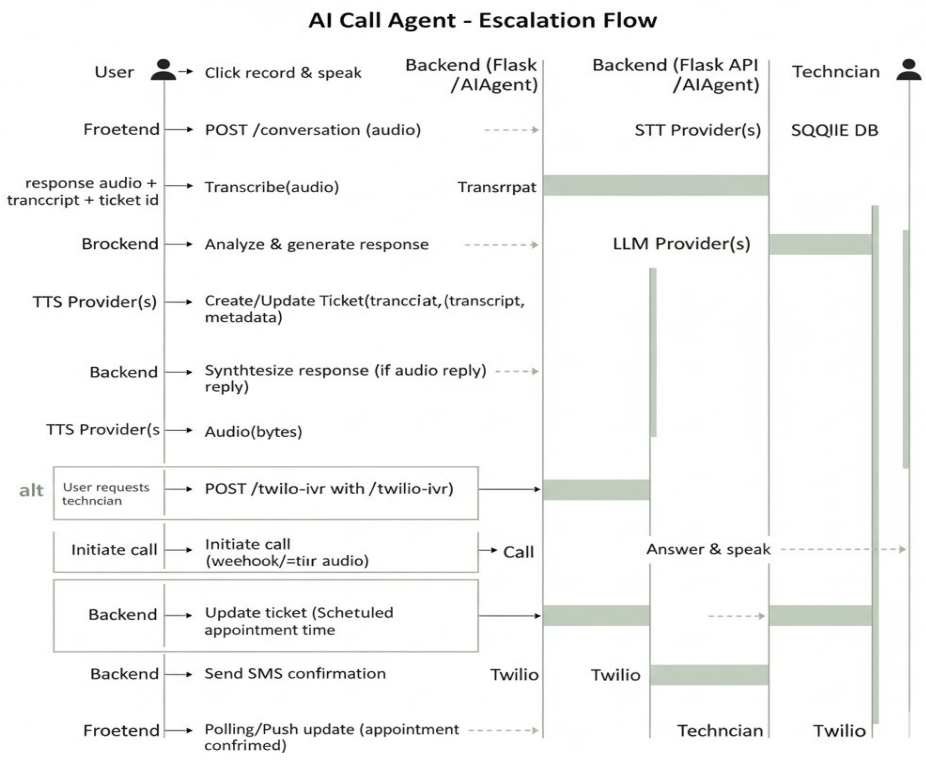

Figure 4.2: Use Case Diagram

4.3 Activity Model

The system has a few important things it can do. The first one, UC-01: Submit Problem via Voice, happens when a user presses the record button and speaks about their issue. After the user logs in, the system changes their speech into text, creates a support ticket, and starts checking what the problem might be.

The second one, UC-02: Automated Diagnosis, is handled fully by the AI Agent. The system talks with the user in a back-and-forth way, asks follow-up questions, and suggests basic fixes. If the issue can be solved, the system does it. If not, it prepares to send the issue to a human technician.

The third one, UC-03: Escalate and Schedule Technician, is used when the AI decides a human expert is needed. The system automatically calls a technician through an IVR call, explains the problem, asks for their available time, converts their voice reply into text, and then books an appointment in the ticket.

Next, UC-04: Technician Confirmation via SMS kicks in. After scheduling, the system sends a text message to the technician to confirm the appointment details.

Finally, UC-05: Admin Manages Technicians and Logs is for the administrator. From the admin dashboard, they can add or edit technician details, update skill sets, and look through call logs or conversation history whenever needed.

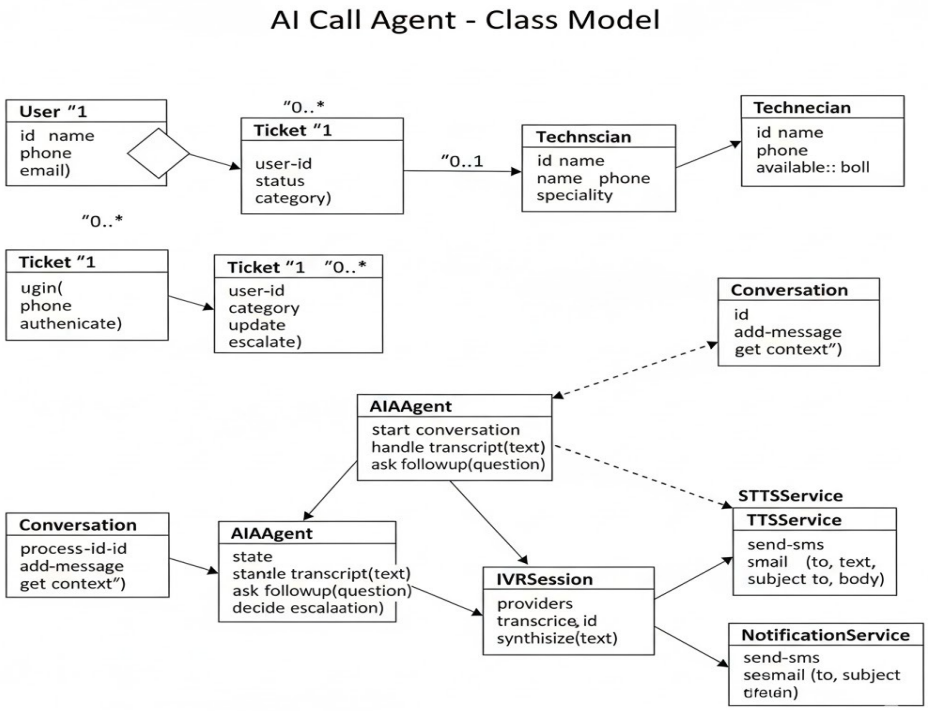

Figure 4.3: Activity Model

Chapter 5

IMPLEMENTATION AND TESTING

5.1 Implementation

The implementation of the AI Call Agent System involves building a reliable, scalable, and intelligent platform capable of handling real-time voice interactions between users and technicians. The system integrates a secure backend powered by Python/Flask, which manages speech transcription, natural language understanding, ticket creation, IVR operations, and technician scheduling. The frontend is developed as a browser-based Single Page Application (SPA) that supports audio capture, playback of synthesized responses, ticket status display, and user authentication.

A modular architecture is adopted to ensure smooth interaction across components such as Speech-to-Text (STT), Large Language Models (LLM), Text-to-Speech (TTS), and Twilio telephony services. The backend uses a provider-fallback mechanism to maintain reliability even when individual STT/TTS/LLM services fail. The system also incorporates secure database storage using SQLite or PostgreSQL, with models for tickets, technicians, appointments, call logs, and conversation history.

Security measures such as HTTPS endpoints, environment-based key management, authentication tokens, and role-based access control are implemented to protect sensitive information. The application is deployed on a secure server, and continuous monitoring ensures smooth operation, updates, and improvements based on user feedback.

5.2 Testing

Testing ensures that the AI Call Agent system meets the expected requirements and performs reliably during real-time communication. Various testing methods are applied to examine each functional component and verify that the overall system works cohesively. The testing process includes evaluating accuracy, call flow correctness, response timing, reliability of external providers, and proper handling of edge cases.

Both automated and manual testing approaches are used to identify defects, validate workflow correctness, and ensure that fallback logic behaves as expected when providers fail. The system is tested under different network conditions to confirm its robustness, especially during IVR calls and live audio communication.

5.2.1 Unit Testing

Unit testing focuses on independently checking each module of the AI Call Agent to ensure it performs as intended. Individual components—such as the STT module, TTS engine, LLM prompt handler, ticket manager, and Twilio webhook processor—are tested in isolation. This ensures that even the smallest units of the application behave correctly under a variety of inputs.

Since many of these modules involve audio, text processing, and API interaction, mock services and simulated data are used during testing. Unit testing follows a White-Box testing approach, as internal logic such as intent extraction, provider fallback, and dialogue state transitions must be verified in detail. The successful execution of unit tests provides confidence that all independent modules function correctly before integrating them into the full system.

5.2.2 Integration Testing

When the different modules of the AI Call Agent system are combined, Integration Testing becomes essential. This stage ensures that the STT engine, LLM-based AI Agent, TTS provider, Twilio IVR system, backend APIs, and frontend SPA all work smoothly with one another. Integration testing is carried out after unit testing and focuses on verifying how well modules interact at their boundaries.

During testing, issues often arise when one component passes unexpected data to another—such as inaccurate speech transcripts, missing intent values, or delayed Twilio webhook responses. These problems help identify interaction defects that might not appear during isolated module testing. Integration testing in this system uses both Black Box and White Box approaches, as some tests require internal logic validation (e.g., AI Agent state transitions), while others validate only the external behavior (e.g., correct IVR call flow). This process ensures that the entire communication chain works consistently and reliably under real-time conditions.

5.2.3 System Testing

System Testing evaluates the AI Call Agent as a complete product to ensure that the entire workflow behaves as expected. At this level, the fully integrated system—including the frontend, backend, databases, AI components, and telephony services—is tested end-to-end. The goal is to confirm that the entire support flow (voice input $\rightarrow$ diagnosis $\rightarrow$ escalation $\rightarrow$ technician scheduling $\rightarrow$ SMS confirmation) works correctly as a whole.

Various system tests are performed, such as functionality testing to validate correct ticket creation and escalation behavior, performance testing to measure response time during peak loads, and reliability testing to ensure provider fallback works when a service fails. Portability testing is also conducted to verify system behavior across different browsers, devices, and network conditions.

Chapter 6

Results and Analysis

The AI Call Agent system was successfully implemented and tested, demonstrating that the platform meets the core requirements of automated voice-based support. The system was able to capture user audio from the browser or phone call, process it through Speech-to-Text services, and generate meaningful responses using the AI Agent and LLM modules. During testing, the agent accurately identified user intent in most cases and created support tickets without delay. The frontend also performed smoothly, with clear audio playback, real-time updates, and a user-friendly interface.

The automated diagnosis feature worked effectively, enabling the AI Agent to guide users through basic troubleshooting steps. In many test scenarios, the AI was able to resolve the issue without human intervention, showing the strength of the dialogue flow and follow-up questioning process. When escalation was necessary, the system reliably selected an appropriate technician based on skill and availability and correctly transitioned into the scheduling workflow.

The escalation and IVR interaction through Twilio also produced successful results. Test calls were placed consistently, the technician received a clear summary of the issue, and the system was able to capture the technician's availability using voice input. Extracted time and date values were correctly interpreted in most cases, and the ticket status was updated to Scheduled as expected. After scheduling, technicians received confirmation SMS messages without errors, completing the cycle smoothly.

Overall, the system performed reliably across all functional areas, including ticket creation, automated diagnosis, IVR call handling, appointment scheduling, and SMS confir-

Fig 6.1 This figure displays the account creation page used to register new users in the AI Call Agent system. The form captures a username and password and allows the user to switch back to the sign-in page.

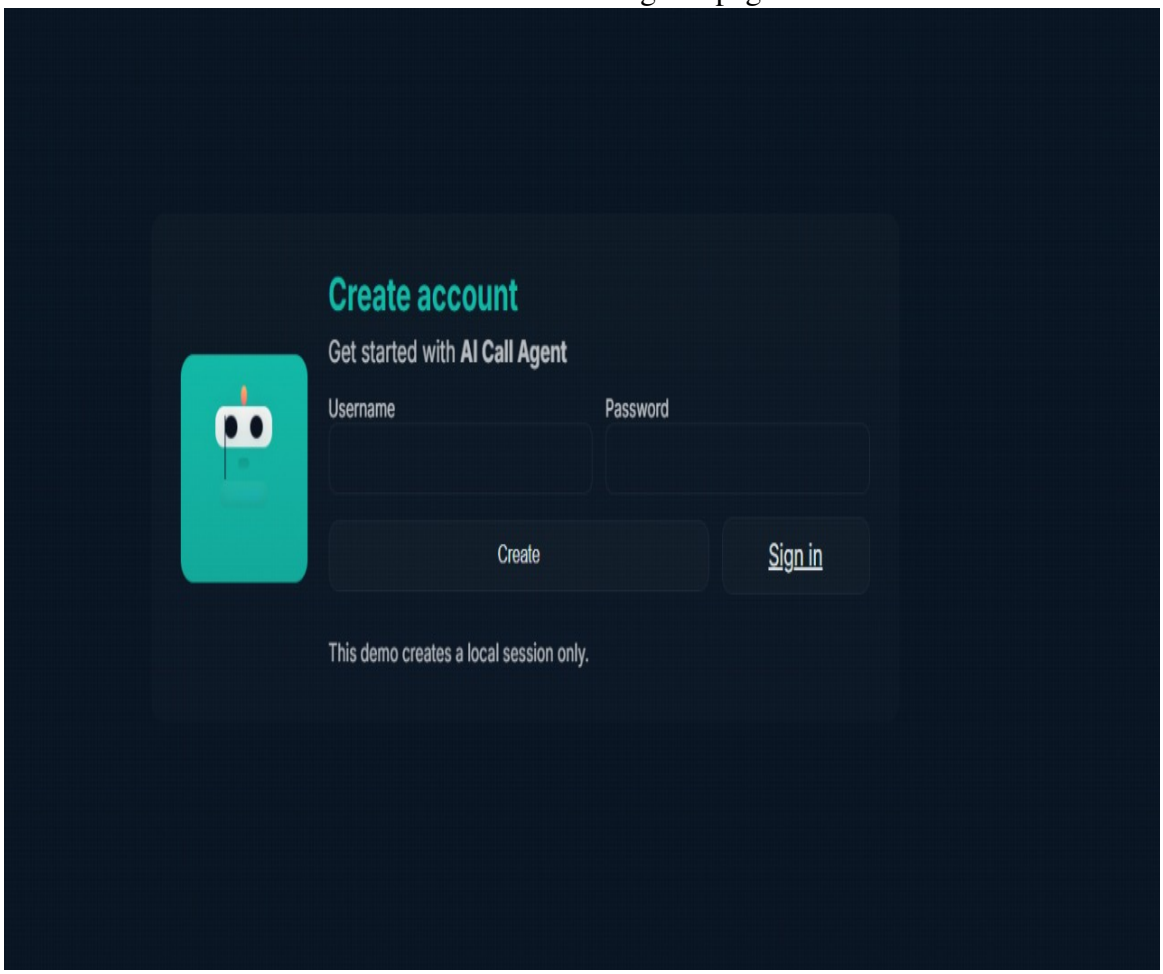

Figure 6.1: Create Account Page

mation. Minor issues such as unclear speech or noisy environments were handled through fallback logic, ensuring uninterrupted workflow. The results confirm that the AI Call Agent is stable, efficient, and ready for practical deployment.

Fig 6.2 This figure shows the login screen of the AI Call Agent system, where users enter their username and password to access the dashboard. The interface also includes an option to create a new account if the user is not registered.

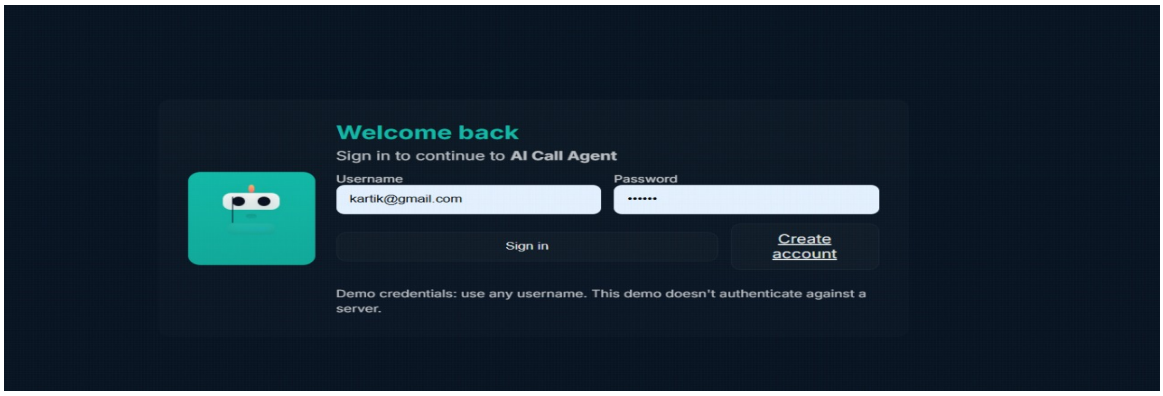

Figure 6.2: Login Page – User Authentication Screen

Fig 6.3 This figure shows the main dashboard of the AI Call Agent system after the user logs in. It displays the system status, the conversation workflow stages (Describe, Diagnostics, Urgency, Calling Tech, Result), and the microphone button to start voice input. The sidebar provides quick actions, recent conversations, and technician management options.

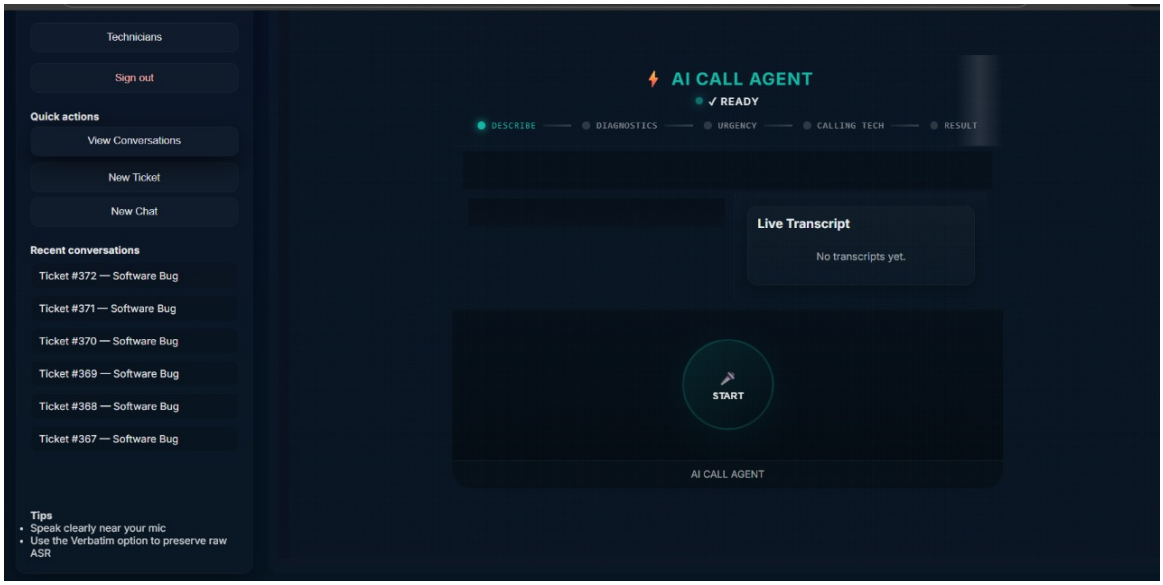

Figure 6.3: AI Call Agent – Home Dashboard Interface

Fig 6.4 This figure shows the administrator interface used for managing technicians. The left section allows adding new technicians with details such as name, phone number, problem type, skillset, and availability. The right section lists the existing technicians along with options to edit or delete their profiles.

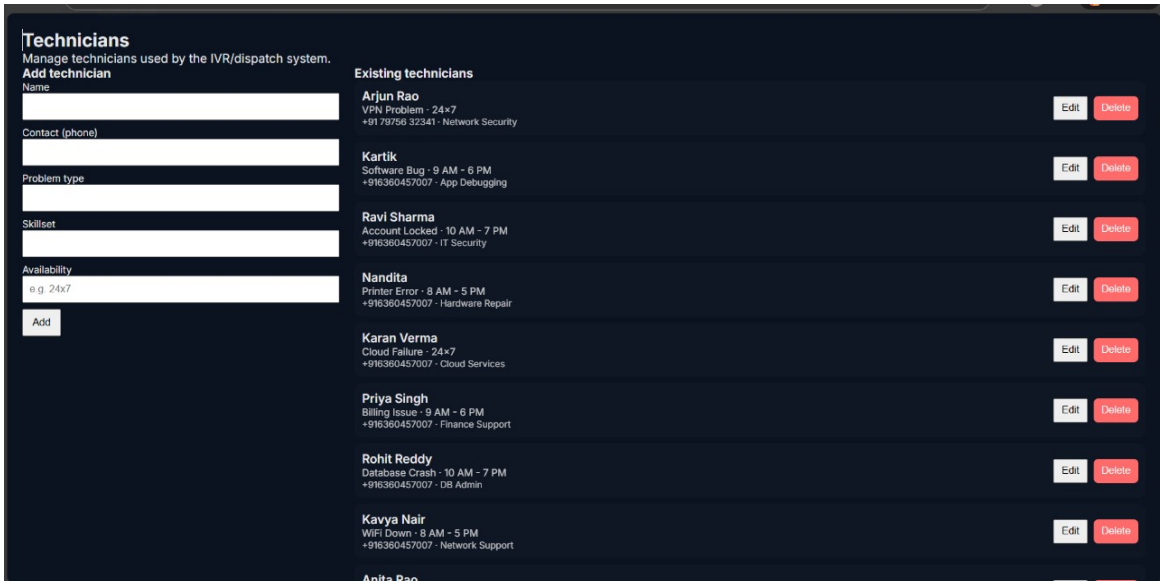

Figure 6.4: Technician Management Panel

Fig 6.5 This figure displays the detailed conversation history for a selected ticket. It shows the agent's messages, technician responses received through IVR, and the final appointment confirmation. The left panel lists event logs such as IVR greeting, time collection, confirmation, and SMS sent notifications.

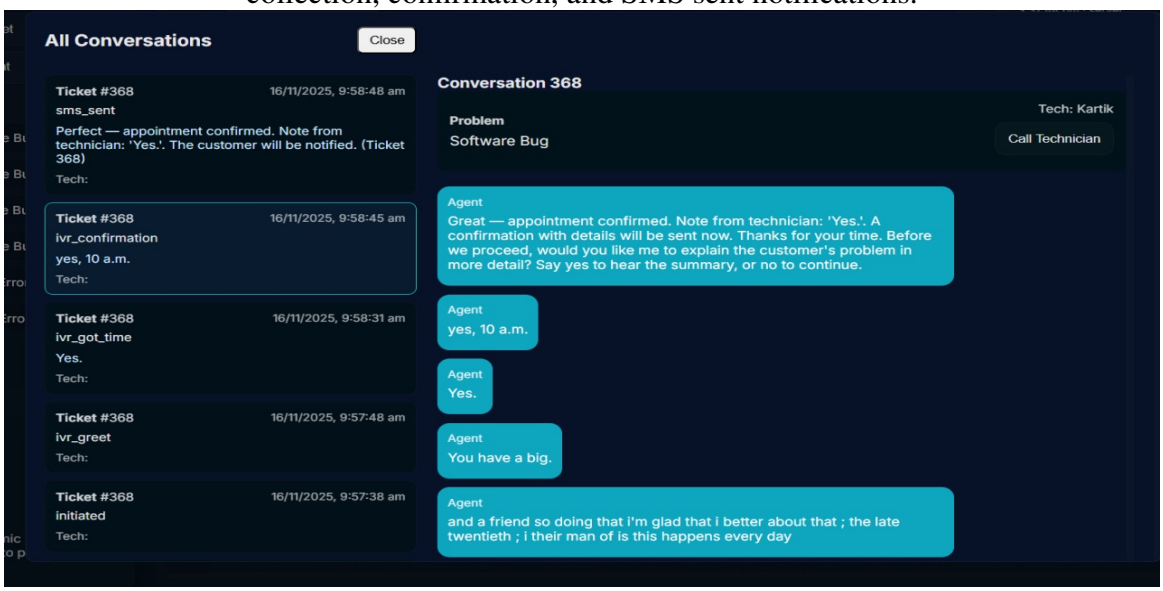

Figure 6.5: Conversation View–Ticket Interaction Details

Fig 6.6 This figure shows the AI Call Agent interface during an active voice conversation while recording is in progress. The AI Agent welcomes the user, identifies the issue as a Software Bug, and prompts for additional details. On the right, the Live Transcript panel displays real-time speech-to-text output captured from the user's microphone. The progress bar above shows the diagnostic stages (Describe, Diagnostics, Urgency, Calling Tech, Result), indicating that the system is currently in the initial conversation phase.

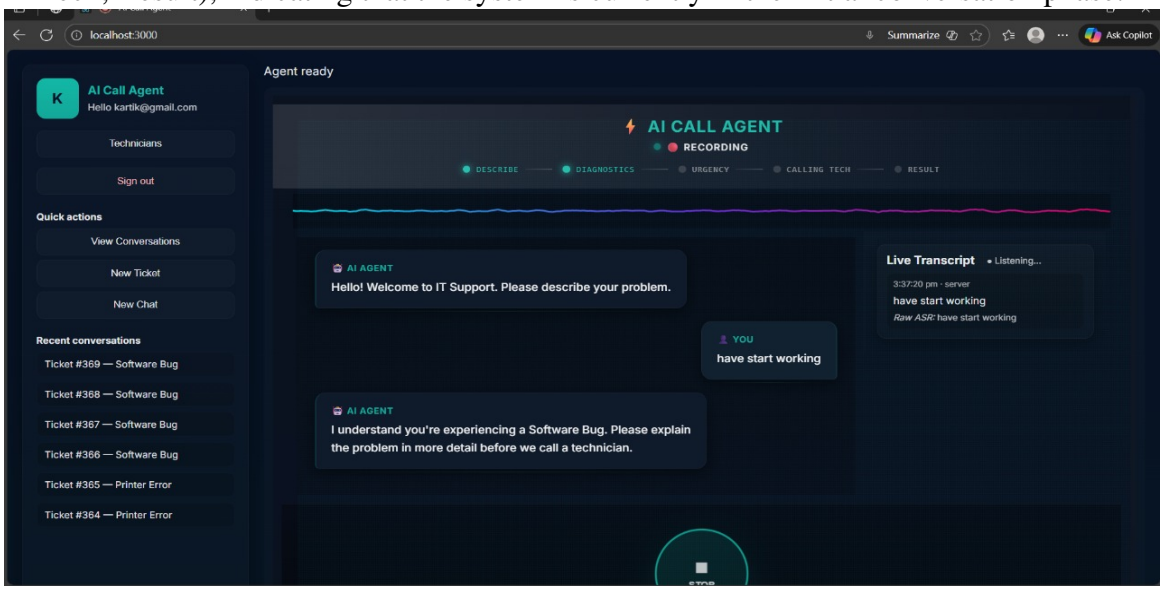

Figure 6.6: AI Call Agent –Conversation and Live Transcription

Chapter 7

Conclusion

The AI Call Agent system is all about having a comprehensive, modern, and scalable solution that will automate first-line IT support using voice-driven interactions. Its well-structured architecture combines STT, TTS, NLP, LLMs, and telephony services in a way that easily supports conversations with users and technicians. Its modular design allows flexibility and maintainability due to its pluggable provider model, robust IVR workflow, and clean frontend–backend separation.

The use case and activity models demonstrate a seamless end-to-end workflow: from the capture of user issues, intelligent diagnosis, escalation when needed, and technician scheduling coordination by means of Twilio IVR. This system is engineered with strong emphasis on reliability, security, scalability, and usability in view to effectively support real-world IT operations.

In all, this design provides a solid basis for a support platform powered by AI to reduce human workload, speed up response times, and improve the user experience through natural voice.

BIBLIOGRAPHY

[1] John Luther **"AI Call Agent "**ISSN No: 2581 - 4230 VOLUME 10, Date of issue: 6, June - 2020.

[2] Smith Seth **"Smart Call Routing "** Vol. 17(2) 167–182 Date of issue: January 5 2019

[3] Lee Jiyeon **"Real-time Conversation Management "**ISSN 0973-1873 Volume 13, Date of issue: Number 3 (2021)

[4] Kumar Gaurav **"Service System Efficiency "** Date of Issue: August 2, 2018.

[5] S.R.Bharamagoudar, Geeta R.B, S.G.Totad. **"AI in Customer Support System"** Date of issue: 6 June 2020.

[6] Brown **"Automated Call Handling "**ISSN 0875-7865 Volume 15, Date of issue: Number 4 (2019)

[7] Wilson **"AI-driven Feedback Analysis"**ISSN 0934-1886 Volume 10, Date of issue: Number 14 (2021)

[8] Patel **"Intelligent Call Distribution "**ISSN 0435-1867 Volume 19, Date of issue: Number 15 (2018)

[9] Zhang **"Conversational AI for Escalation Prediction "**ISSN 0967-1856 Volume 19, Date of issue: Number 10 (2020)

[10] Fernandez **"Multilingual AI Support System "**ISSN 0945-1865 Volume 9, Date of issue: Number 8 (2021)